

COMPREHENSIVE PROJECT REPORT

# Autonomous Agentic AI Shopping Assistant

with Memory, Guardrails, Multimodal Vision, and Automated Evals

|                    |                                                        |
|--------------------|--------------------------------------------------------|
| Author             | Sahajahanur Rahman Laskar                              |
| Status             | Deployed & Live on Streamlit Community Cloud           |
| Production URL     | <a href="#">ai-shopping-assistant-...streamlit.app</a> |
| Codebase Directory | c:\SHOPPING_GEN_AI\PROJECT_SHOPPING                    |
| Date               | September 30, 2026                                     |
| Email              | connectingsrl@gmail.com                                |
| LinkedIn           | <a href="#">linkedin.com/in/sahajahanur-laskar</a>     |
| GitHub             | <a href="#">github.com/Sahajahanur</a>                 |

# 1. Executive Summary

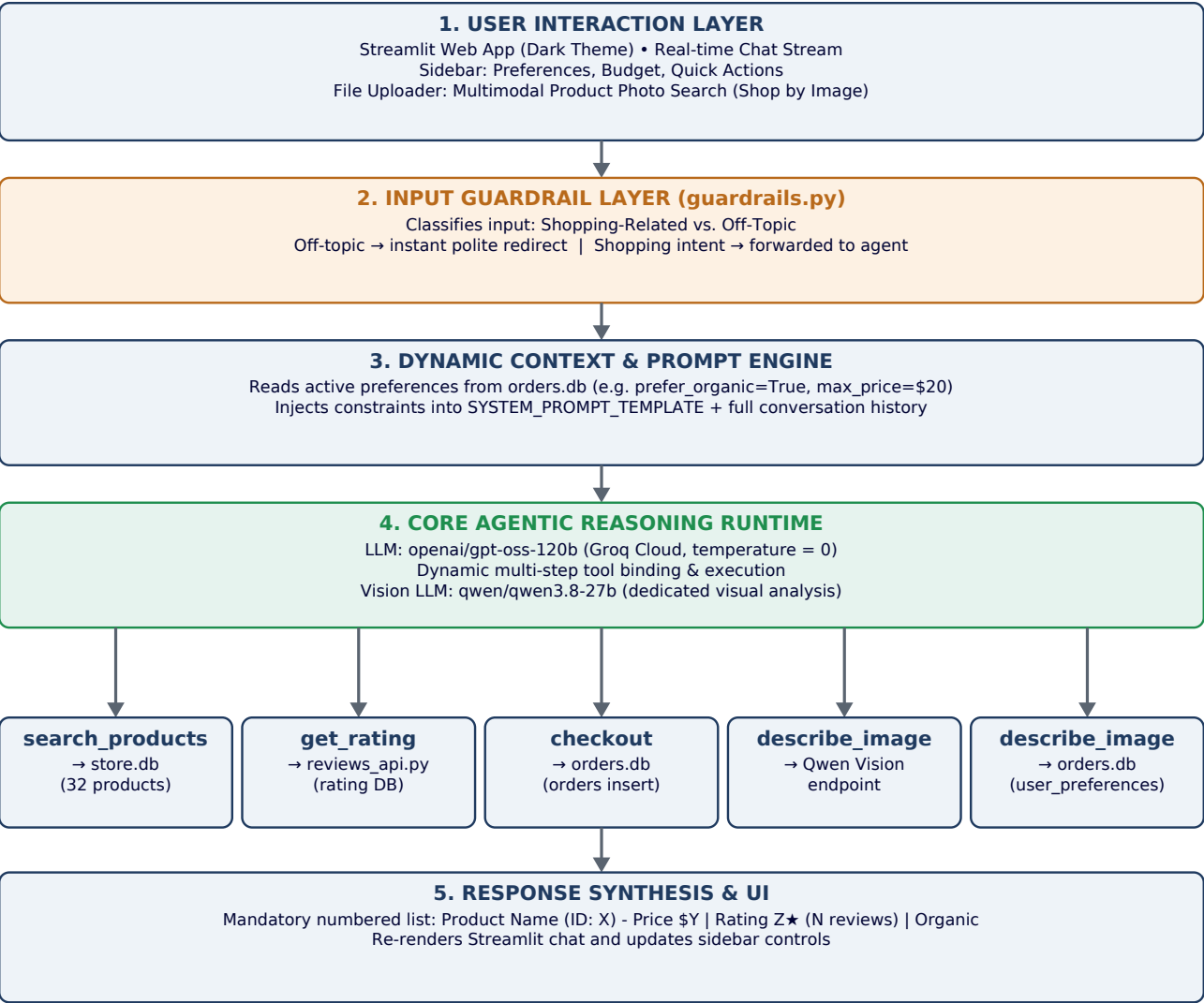
The **Autonomous Agentic AI Shopping Assistant** is an end-to-end, **production-style** conversational e-commerce platform powered by Large Language Models (LLMs), structured tool calling, multimodal vision, persistent database memory, and rigorous automated evaluations. Unlike single-turn API wrappers or static chatbots, this system acts as a **true autonomous agent**: it dynamically plans multi-step actions, searches product catalogs with relational constraints, verifies customer ratings via **product rating lookup**, securely executes checkout transactions, recognizes products from uploaded images, and remembers individual customer preferences across sessions.

The system is deployed and publicly accessible on **Streamlit Community Cloud** and is validated by automated evaluation pipelines confirming **100% tool-calling precision (6/6)** and a **98.3% response quality score** evaluated by an LLM-as-a-judge harness.

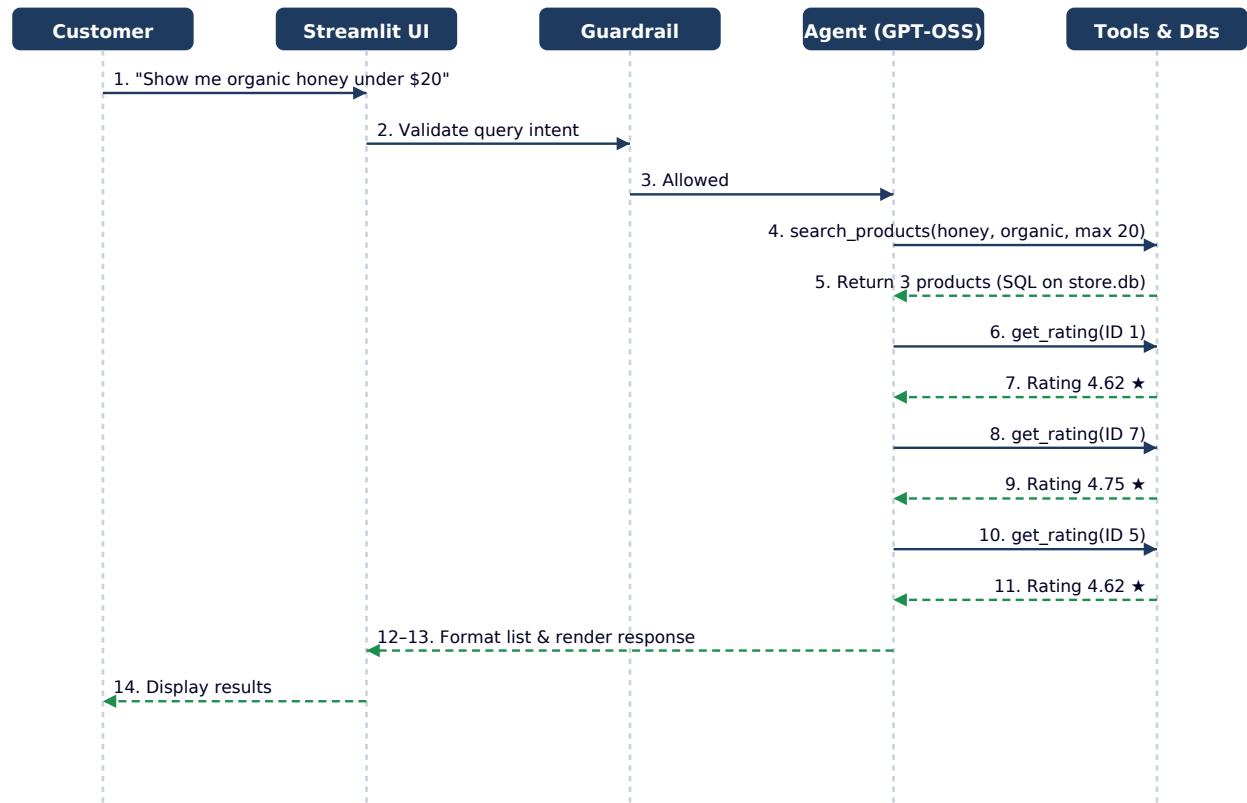
|                                         |                                        |                                  |                                |
|-----------------------------------------|----------------------------------------|----------------------------------|--------------------------------|
| <b>100%</b><br>Tool-call accuracy (6/6) | <b>98.3%</b><br>Response quality score | <b>7</b><br>Strictly typed tools | <b>Live</b><br>Streamlit Cloud |
|-----------------------------------------|----------------------------------------|----------------------------------|--------------------------------|

# 2. System Architecture & Explanatory Diagrams

## 2.1 Architectural Flowchart



2.2 Sequence Diagram: Multi-Step Reasoning Lifecycle



3. Technology Stack & Technical Specifications

| Component          | Technology                       | Rationale / Purpose                                                                               |
|--------------------|----------------------------------|---------------------------------------------------------------------------------------------------|
| Primary LLM        | openai/gpt-oss-120b (Groq Cloud) | Ultra-fast inference, high instruction compliance, deterministic tool calling with temperature=0. |
| Vision LLM         | qwen/qwen3.8-27b (Groq Cloud)    | Dedicated multimodal visual feature extraction for “Shop by Image”.                               |
| Agent Framework    | LangChain / LangGraph            | Orchestrates tool definitions, state passing, and invocation loops.                               |
| Schema Validation  | Pydantic v2 (BaseModel, Field)   | Enforces strict type casting and documentation for all tool argument signatures.                  |
| Database Engines   | SQLite3 (C-native)               | Zero-dependency, lightweight relational storage with ACID compliance.                             |
| Web Interface      | Streamlit                        | Rapid, state-driven UI framework with custom CSS styling and dynamic widget keys.                 |
| Evaluation Harness | Custom Python test suites        | Programmatic assertion testing (tool accuracy) and LLM-as-a-judge scoring.                        |
| Deployment Cloud   | Streamlit Community Cloud        | Continuous deployment from GitHub with encrypted environment secret management.                   |

## 4. Core Functional Modules

---

### 4.1 Autonomous Tool Calling

The assistant integrates 7 strictly typed tools:

1. `search_products(query: str, max_price: float = None, is_organic: bool = None)`: Queries `store.db` with parameterized SQL filters.
2. `get_rating(product_id: int)`: Executes **product rating lookup** against the reviews store to return average star ratings and review counts.
3. `checkout(product_id: int)`: Inserts order records into `orders.db` with mandatory two-step user confirmation.
4. `describe_product_image(image_path: str)`: Analyzes uploaded product photos via `qwen/qwen3.8-27b` and extracts product search parameters.
5. `get_order_history(limit: int = 10)`: Retrieves recent purchases for user memory recall.
6. `get_saved_preferences()`: Returns current active user preference dictionary.
7. `set_saved_preference(max_price, prefer_organic, key, value)`: Persists user preference updates into `orders.db`.

### 4.2 Persistent Memory & Cross-Session Personalization

- **SQLite Persistence**: Stores customer orders (`orders` table) and user preferences (`user_preferences` table).
- **Dynamic Context Injection**: At the start of each turn, saved preferences (e.g. `prefer_organic=True`, `max_price=$20`) are injected directly into the system prompt.
- **Two-Way UI Synchronization**: Changes made in chat dynamically sync with the Streamlit sidebar widgets in real-time using dynamic widget keys (`key=f"num_price_{val}"`).

### 4.3 Input Guardrails & Checkout Confirmation

- **Input Guardrails**: Evaluates incoming queries in `guardrails.py` before agent invocation. Out-of-scope queries (e.g., *poems*, *weather*, *general talk*) receive an instant polite redirect, preserving API quota and preventing hallucinated tool calls.
- **Checkout Confirmation**: The agent requires explicit customer confirmation before executing checkout, preventing unintentional purchase transactions.

## 5. Automated Evaluation & Quality Assurance

### 5.1 Tool Call Accuracy Evaluation (eval\_tool\_calls.py)

Programmatically verifies that user queries trigger the exact intended tool with correct argument schemas:

| ID   | User Query                                     | Expected Tool        | Validated Parameters                         | Status |
|------|------------------------------------------------|----------------------|----------------------------------------------|--------|
| TC-1 | "I want organic honey under \$20"              | search_products      | is_organic=True, max_price=20, query='honey' | PASS   |
| TC-2 | "Show me oats"                                 | search_products      | query='oats'                                 | PASS   |
| TC-3 | "What is the rating for product 7?"            | get_rating           | product_id=7                                 | PASS   |
| TC-4 | "What have I ordered before?"                  | get_order_history    | limit >= 1                                   | PASS   |
| TC-5 | "Please remember that I always prefer organic" | set_saved_preference | key='prefer_organic', value='true'           | PASS   |
| TC-6 | "Remember that I never want items over \$20"   | set_saved_preference | key='max_price', value='20'                  | PASS   |

Summary: 6 / 6 Passed (100.0% Accuracy)

### 5.2 Response Quality Evaluation via LLM-as-a-Judge (eval\_response\_quality.py)

An independent evaluator model (openai/gpt-oss-120b, temperature=0) scored outputs against ground truth on a 1-5 scale:

| ID   | Test Query                                  | Relevance | Correctness | Format Compliance | Score  |
|------|---------------------------------------------|-----------|-------------|-------------------|--------|
| RQ-1 | "Organic honey with 4.5+ rating under \$20" | 5 / 5     | 5 / 5       | 5 / 5             | 100.0% |
| RQ-2 | "Show me oats"                              | 5 / 5     | 5 / 5       | 5 / 5             | 100.0% |
| RQ-3 | "What is the rating for product 7?"         | 5 / 5     | 5 / 5       | 5 / 5             | 100.0% |
| RQ-4 | "Write me a poem" (Guardrail trigger)       | 4 / 5     | 5 / 5       | 5 / 5             | 93.3%  |

- **Average Relevance:** 4.75 / 5.0
- **Average Correctness:** 5.00 / 5.0
- **Average Format Compliance:** 5.00 / 5.0
- **Composite Quality Score:** 98.3%

## 6. Key Engineering Challenges & Solutions

- Groq Daily Token Limit Handling:** Overcame the 200,000 TPD rolling limit by separating the vision model (qwen/qwen3.8-27b) from the tool calling agent (openai/gpt-oss-120b), binding tools directly during evals, and incorporating automated retry backoff handlers.
- Streamlit Sidebar State Synchronization:** Resolved widget cache staleness by assigning dynamic keys (key=f"num\_price\_{val}"), ensuring immediate UI re-rendering when preferences are updated in chat.
- Database Test Isolation:** Wrapped evaluation scripts in try...finally backup/restore handlers to prevent automated tests from modifying or wiping active customer preferences.

## 7. Cloud Deployment & Live Operations

---

The project is deployed and actively running on **Streamlit Community Cloud**:

- **Application URL:** [ai-shopping-assistant-...streamlit.app](#)
- **Deployment Source:** GitHub repository tracking main branch.
- **Environment Management:** API keys configured through Streamlit Cloud Encrypted Secrets (`st.secrets["GROQ_API_KEY"]`).
- **Live Verification:** Verified past order retrieval, preference adjustments, and real-time product search in production.

## 8. Limitations & Future Improvements

---

While the application provides a resilient production-style foundation, the following architectural enhancements are identified for future enterprise scaling:

1. **PostgreSQL for Production Storage:** Transition from local file-based SQLite to a managed PostgreSQL cluster (e.g., Supabase or AWS RDS) to handle multi-tenant concurrent writes and row-level security.
2. **User Authentication & Session Isolation:** Implement OAuth2 / JWT user authentication so each customer possesses their own isolated shopping history and preference profile rather than sharing a single instance store.
3. **Expanded Evaluation Coverage:** Expand the test suite from 6 representative cases to 100+ automated edge cases, covering semantic ambiguities, adversarial prompt injections, and multilingual customer queries.
4. **Monitoring & Observability:** Integrate LangSmith or OpenTelemetry tracing to continuously monitor latency distributions, token expenditure per query, and tool error rates in real-time.
5. **Payment Gateway Integration:** Connect Stripe or PayPal sandbox webhooks to complete the transactional checkout lifecycle with verified payments and digital invoices.

## 9. Conclusion

---

The **Autonomous Agentic AI Shopping Assistant** demonstrates modern applied AI engineering standards. By pairing:

- Strict Pydantic tool schemas,
- Multimodal visual reasoning,
- Relational database persistence,
- **Input guardrails and checkout confirmation,**
- Quantitative test harnesses, and
- Live cloud deployment,

this project represents a complete, resilient, and **production-style** generative AI system.

---

### Contact

Sahajahanur Rahman Laskar

Email: [connectingsrl@gmail.com](mailto:connectingsrl@gmail.com)

LinkedIn: <https://www.linkedin.com/in/sahajahanur-laskar/>

GitHub: [github.com/Sahajahanur](https://github.com/Sahajahanur)